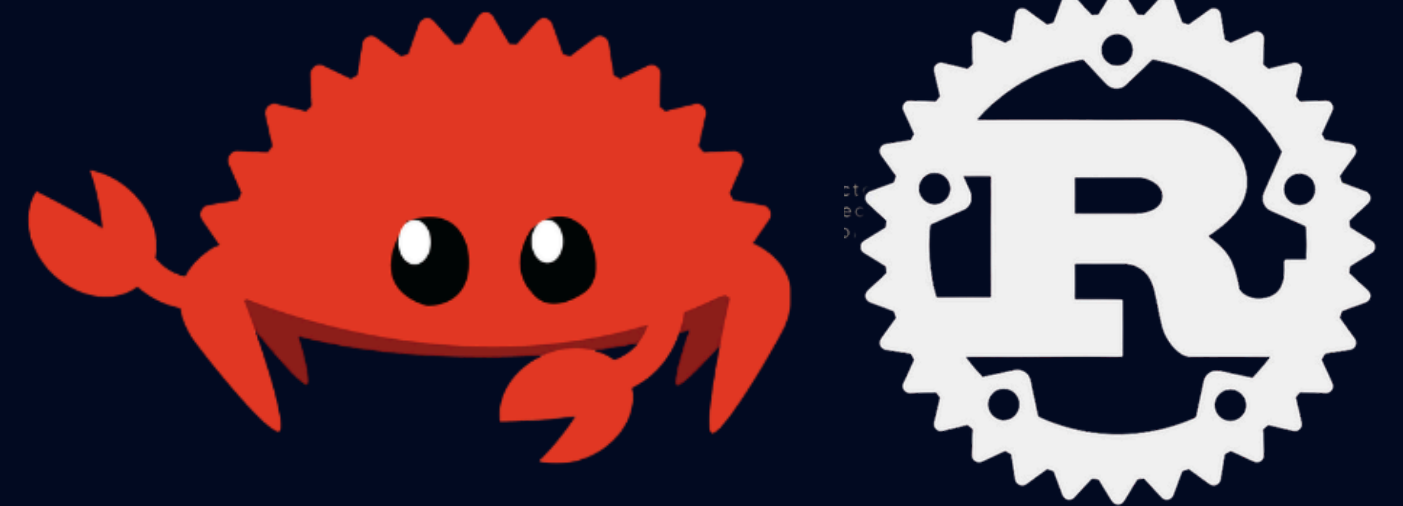


Secure Digital Signature using WebAssembly with **Rust**

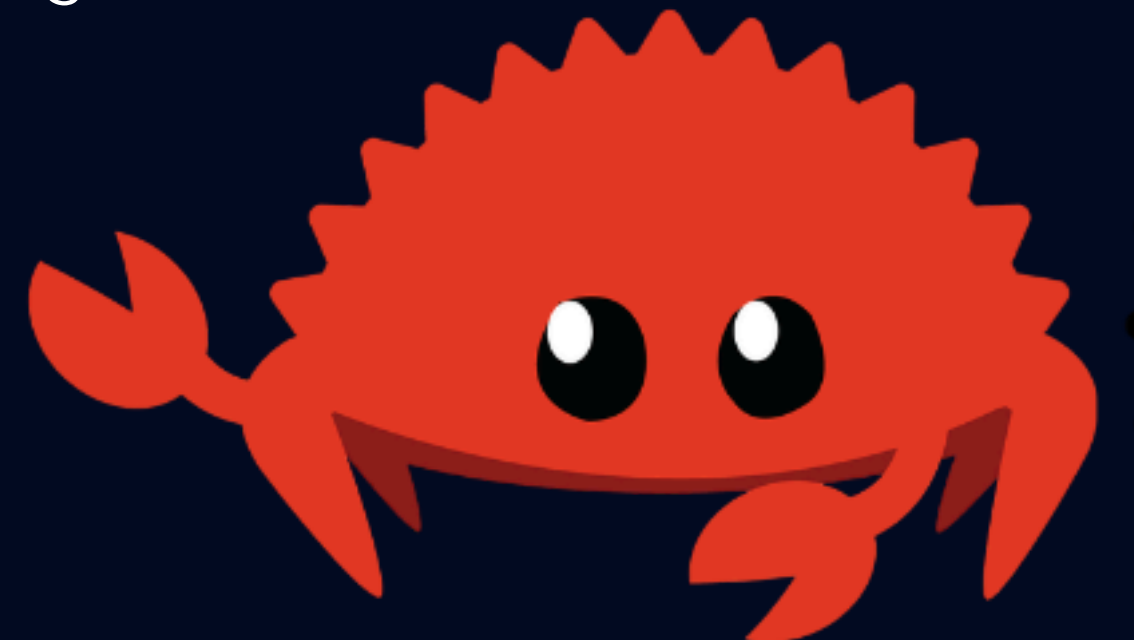


presentado por: **XIMENA & FER**

¿Qué es **Rust** ?



Lenguaje de programación compilado, que se caracteriza por su sistema de manejo de memoria basado en reglas estrictas de **ownership**, **borrowing** y **lifetime**, que permiten garantizar la seguridad de la memoria en tiempo de compilación sin necesidad de un Garbage Colector



Webassembly

¿ Qué es Webassembly ?

Es un formato binario portable y de bajo nivel diseñado para ejecutar código de alto rendimiento dentro del navegador.

Aplicaciones

Criptografía

- Frimas digitales, hash, cifrado

Videojuegos web

- Motores o lógica pesada en navegador

Procesamiento de archivos

- Leer, transformar o validar archivos

Simuladores

- Física, matemática, cálculos rápidos

Importancia de WASM + Rust

- Permite trabajar datos confidenciales localmente en el navegador.
- Evita enviar información privada (llave privada) al servidor, ayudando a reducir el riesgo de exponer datos privados durante procesos.
- Comparte solo el resultado final.
- Combina interfaz web con logica segura en rust.

Main.rs VS Lib.rs

	Main.rs	Lib.rs
Funcion	Se usa para ejecutar directamente en la terminal	Se usa para crear funciones reutilizables que pueden ser llamadas desde otro lugar de proyectos o programas.
Ejecutar funciones	Se usa la función fn main() y es aquí en donde comienza el programa	JS “llama” a las funciones de Rust desde afuera cuando las necesita
Atributo necesario	No se requiere	Antes de cada función que quieras que JS pueda ver y usar se pone la attribute macro #[wasm_bindgen]
Compilación	<u>Compila y ejecuta</u> 1. cargo run <u>Compila y ejecuta por separado</u> 1. cargo build (compila) ./target/debug/nombre_del_proyecto (ejecuta manualmente)	<u>Compila y ejecuta por separado</u> 1. Compila con: cargo build -release -target wasm32 unknown-unknown -release crea un archivo mas optimizado -target wasm32 código se compila en WASM de 32 bits y no en tu computadora. unknown sistema (Apple,Windows,Linux,etc) -unknown sistema operativo o entorno en donde el programa correrá. 2. Ejecuta con: wasm-bindgen --target web --out-dir web/pkg target/wasm32-unknown-unknown/release/nombre_proyecto.wasm wasm-bindgen conecta Rust/WASM con JS -target web indica que usaremos una pagina web -out-dir web/pkg indica la carpeta en donde se guardan los archivos generado target/wasm32... ruta del archivo .wasm que genero cargo build

WEBASSEMBLY+RUST DEMO

MULTIPLICACION

```
cargo new mult_demo --lib
```

```
vim cargo.toml  
// crate y dependencies
```

```
cd src  
vim lib.rs // modificar archivo
```

cambiar en archivo el nombre del archivo .wasm

```
touch index.html // crear interfaz  
vim index.html //mod archivo
```

linea 229

```
cargo build --release --target wasm32-unknown-unknown  
//compila
```

```
wasm-bindgen \--target web \./target/wasm32-unknown-  
unknown/release/mult_demo.wasm \--out-dirpkg  
//ejecuta
```

```
python3 -m http.server 8000  
//levanta servidor
```

```
http://localhost:8000  
//ponerlo en nav
```


DEMO - SIMULACIÓN PAGO DE IMPUESTOS

Dinámica

Primero, el usuario tendrá un documento que representa su declaración o comprobante de impuestos. Para poder enviarlo, el sistema necesita asegurarse de dos cosas: que realmente lo está enviando esa persona y que el documento no fue modificado. Para eso se usa la llave privada.

Cuando el usuario firma el documento, el sistema genera un hash, que es como una huella digital única del archivo. Después, ese hash se firma con la llave privada del usuario y se guarda en un archivo JSON junto con los datos de la firma.

Luego, en la parte de verificación, se usa la llave pública para revisar la firma. Si la firma coincide con el documento, significa que:

- el documento fue firmado por la persona que tenía la llave privada;
- el archivo no cambió después de ser firmado;
- la información puede considerarse válida para continuar con el trámite.

Criptografía asimétrica

Criptografía asimétrica es un método de seguridad que usa dos llaves diferentes pero relacionadas:

1. La llave privada solo la tiene el dueño y no debe compartirse, porque sirve para demostrar que él fue quien firmó o autorizó algo.
2. La llave pública sí se puede compartir, porque sirve para que otras personas verifiquen que la firma es válida.

Cualquiera puede usar la llave pública para cifrar datos, pero solo aquellos con la llave privada correcta pueden descifrar esos datos para leerlos.

RSA <https://www.cryptool.org/en/cto/rsa-step-by-step/>

Matemáticamente, RSA se basa en dos números primos grandes que se multiplican para crear n . A partir de ellos se generan dos exponentes: e , que forma parte de la llave pública, y d , que forma parte de la llave privada.

Lo importante es que lo que se firma con d solo puede verificarse correctamente con e , por eso RSA permite demostrar autenticidad sin revelar la private key

Instalar para macOS:

```
1) /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
2) brew install python
revisamos:
python3 --version

3) brew install openssl
revisamos:
openssl version

rust:
4) curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
(cerramos terminal)

5)source "$HOME/.cargo/env"
revisamos:
rustc --version
cargo --version
rustup --version

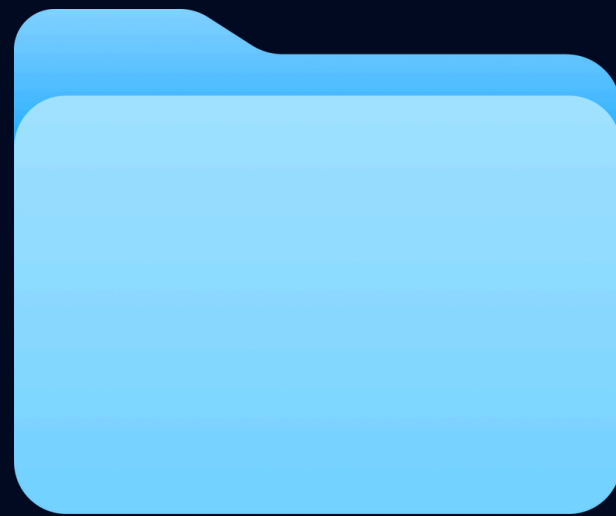
webassembly + wasm-bindgen:
6) rustup target add wasm32-unknown-unknown
revisamos:
rustup target list --installed
(resultado: wasm32-unknown-unknown)

7) cargo install wasm-bindgen-cli
revisamos:
wasm-bindgen --version

8) python3 -m pip install cryptography
revisamos:
python3 -m pip show cryptography
```

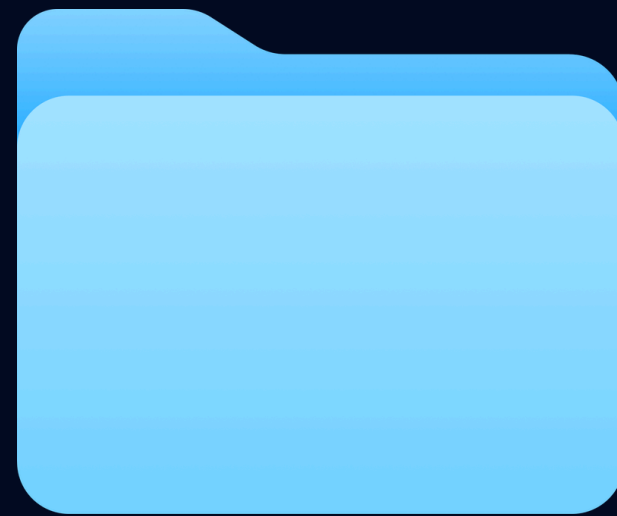
```
brew --version
python3 --version
openssl version
rustc --version
cargo --version
rustup --version
rustup target list --installed
wasm-bindgen --version
python3 -m pip show cryptography
```


firma-demo



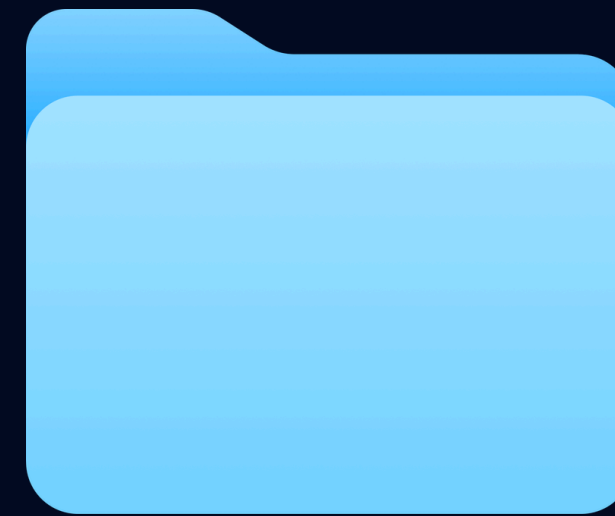
keys

🔒 private_key.pem
🔒 public_key.pem



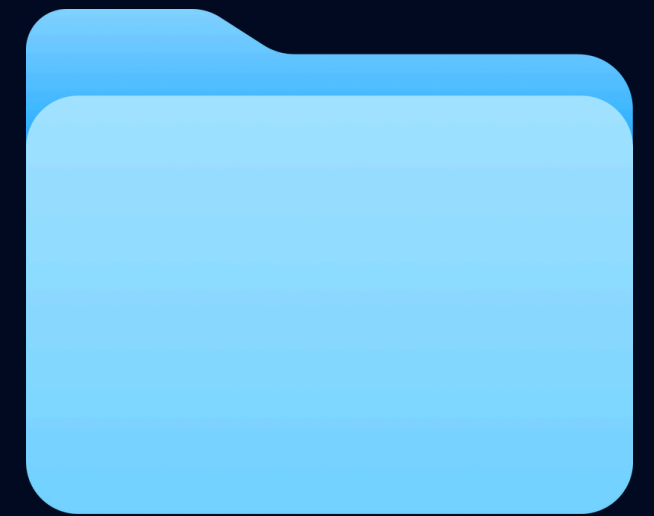
signer_wasm

▾ src
• lib.rs



verify

• index.html
• server.py
• verify_app.js



web

> pkg
• app.js
• index.html

crear llaves:

dentro de la carpet a de keys

```
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
```

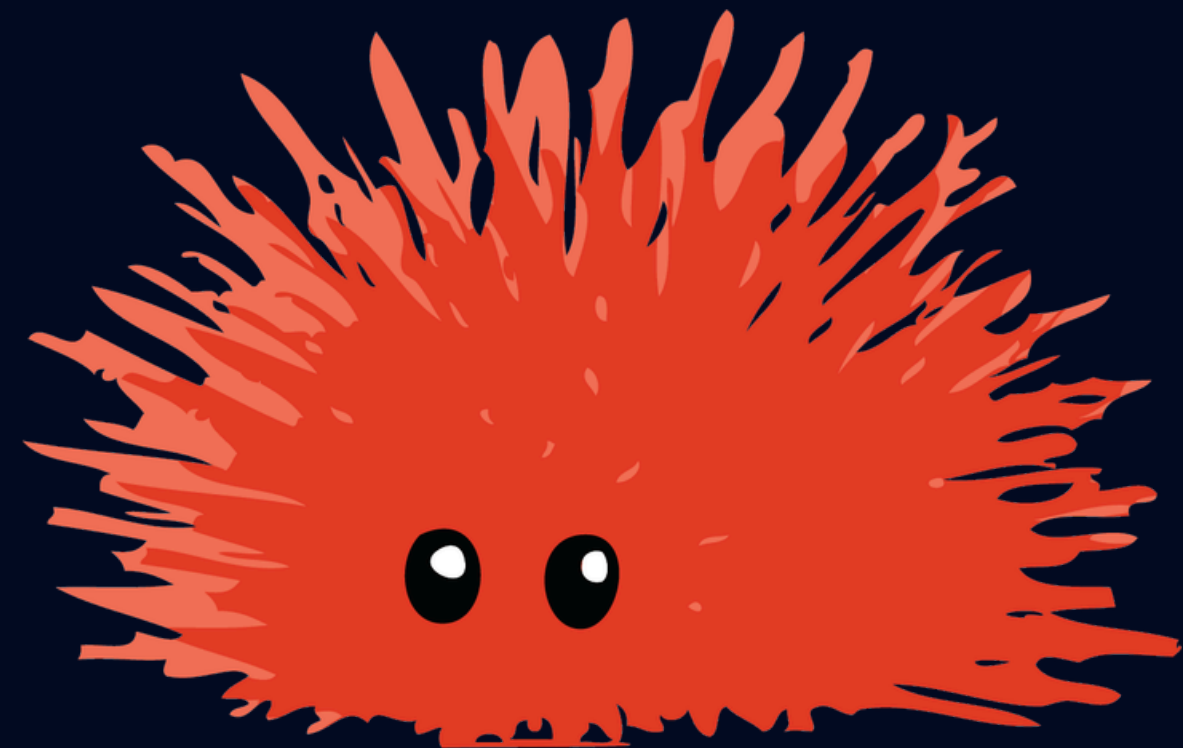
```
openssl pkey -in private_key.pem -pubout -out public_key.pem
```

crear signer_wasm

dentro de la carpeta de demo-showcase

```
cargo new signer_wasm --lib
```

- Rust organiza proyectos con Cargo
- Cargo.toml define el proyecto y dependencias
- src/lib.rs guarda la lógica principal de la librería
- esta librería luego se puede compilar a .wasm
- WASM y wasm-bindgen toman esa librería y la conectan con JavaScript



Cargo.toml

`[package]`: aquí se describe el proyecto

- nombre de tu crate, la versión de tu proyecto y la edición 2024 de Rust

`[lib]`: le dice a Rust que genere una librería compatible que será usada desde JS

- se va a compilar como una librería, no como un `main`

`[dependencies]`

- `wasm-bindgen`: puente entre rust+wasm y js (verificar con `wasm-bindgen --version`)
- `serde`: convertir structs de Rust a JSON o de JSON a structs de Rust
- `serde_json`: lo hace formato json
- `base64`: firma de bytes a texto
- `sha2`: sirve para calcular el hash del documento antes de firmarlo
- `rsa`: es la librería que permite hacer operaciones con RSA

`[target.wasm32-unknown-unknown.dependencies]`

- solo se activa cuando compilamos a `.wasm`
- permite que `getrandom` use las APIs del navegador para generar números aleatorios seguros.
- sin esto podrían salir errores al compilar a `.wasn` porque Rust no sabe de dónde sacar la aleatoriedad)

`[package]`

```
name = "signer_wasm"
version = "0.1.0"
edition = "2024"
```

`[lib]`

```
crate-type = ["cdylib"]
```

`[dependencies]`

```
wasm-bindgen = "=0.2.118"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
base64 = "0.22"
sha2 = { version = "0.10", features = ["oid"] }
rsa = { version = "0.9", features = ["pem"] }
```

`[target.wasm32-unknown-unknown.dependencies]`

```
getrandom = { version = "0.2", features = ["js"] }
```

escribir lib.rs :

lógica principal de la demo de firma digital con Rust y WebAssembly

```
// 1) Crates
use wasm_bindgen::prelude::*;
use base64::{engine::general_purpose, Engine as _};
use rsa::pkcs1v15::SigningKey;
use rsa::pkcs8::DecodePrivateKey;
use rsa::signature::{Signer, SignatureEncoding};
use rsa::RsaPrivateKey;
use serde::{Deserialize, Serialize};
use sha2::{Digest, Sha256};
```

```
// 2) Struct + Serialize + Deserialize
#[derive(Serialize, Deserialize)]
struct SignedPayload {
    name: String,
    message: String,
    document_hash_hex: String,
    signature_base64: String,
}
```

```
// 3) Borrowing + Mutability
fn bytes_to_hex(bytes: &[u8]) -> String {
    let mut s = String::with_capacity(bytes.len() * 2);

    for b in bytes {
        // format! :es una macro y convierte cada byte a hexadecimal
        s.push_str(&format!("{:02x}", b));
    }

    //regresa el string final con el hash en hexadecimal
    s
}
```



```
// 4) Función exportada a JavaScript
#[wasm_bindgen]
pub fn sign_payload(
    document_bytes: &[u8],
    private_key_pem: &str,
    name: &str,
    message: &str,
) -> Result<String, JsValue> {

    // validaciones básicas (si algo falla se regresa Err)
    if document_bytes.is_empty() {
        return Err(JsValue::from_str("El documento está vacío "));
    }

    if name.trim().is_empty() {
        return Err(JsValue::from_str("El nombre está vacío "));
    }

    if message.trim().is_empty() {
        return Err(JsValue::from_str("El mensaje está vacío "));
    }

    //se lee la private key desde PEM
    let private_key = RsaPrivateKey::from_pkcs8_pem(private_key_pem)
        .map_err(|e| JsValue::from_str(&format!("No se puede leer la private key PEM: {e}")))?;
    let document_hash = Sha256::digest(document_bytes);
    let document_hash_hex = bytes_to_hex(&document_hash);

    // Canonical Payload: texto ordenado que será firmado
    let canonical_payload = format!(
        "name={}\nmessage={}\ndocument_hash_hex={}",
        name.trim(),
        message.trim(),
        document_hash_hex
    );
```

```
let signing_key = SigningKey::<Sha256>::new(private_key);

let signature = signing_key.sign(canonical_payload.as_bytes());

let signature_base64 =
    general_purpose::STANDARD.encode(signature.to_bytes());

// Crea el struct final
let output = SignedPayload {
    name: name.trim().to_string(),
    message: message.trim().to_string(),
    document_hash_hex,
    signature_base64,
};

// Convierte el struct a JSON
serde_json::to_string_pretty(&output)
    .map_err(|e| JsValue::from_str(&format!("No se puede crear el
JSON: {e}")))
}
```

.wasm

dentro de la carpeta de `signer_wasm`

```
cargo build --release --target wasm32-unknown-unknown
```

- compila el proyecto de Rust dentro de la carpeta `signer_wasm`
- se usa el target `wasm32-unknown-unknown` y el resultado ya no es un ejecutable normal para la computadora, sino un archivo `.wasm`
- `--release` genera una versión optimizada del código

```
wasm-bindgen \  
  --target web \  
  --out-dir ../web/pkg \  
  target/wasm32-unknown-unknown/release/signer_wasm.wasm
```

- Este comando toma el `.wasm` que generó Cargo y lo procesa con `wasm-bindgen`
- crea una carpeta `pkg` dentro de `web`, con el archivo `.wasm` final y un archivo JavaScript que permite que la página web pueda importar y usar las funciones de Rust

con extensión Hex Editor de
microsoft se puede ver el `.wasm`

web/pkg/

- |— signer_wasm_bg.wasm
- |— signer_wasm_bg.wasm.d.ts
- |— signer_wasm.d.ts
- |— signer_wasm.js

signer_wasm_bg.wasm

signer_wasm_bg.wasm.d.ts

signer_wasm.d.ts

signer_wasm.js

ejecutar web:

dentro de la carpeta de web

Firma local

en terminal:

```
python3 -m http.server 8002
```

en tu navegador:

```
http://localhost:8002
```

ejecutar verify:

dentro de la carpeta de verify

EN TERMINAL 1: recibe json verificado y la llave publica para verificar que coincide con el hash

```
python3 server.py
```

EN TERMINAL 2: verifica firma

```
python3 -m http.server 8003
```

en tu navegador::

```
http://localhost:8003
```